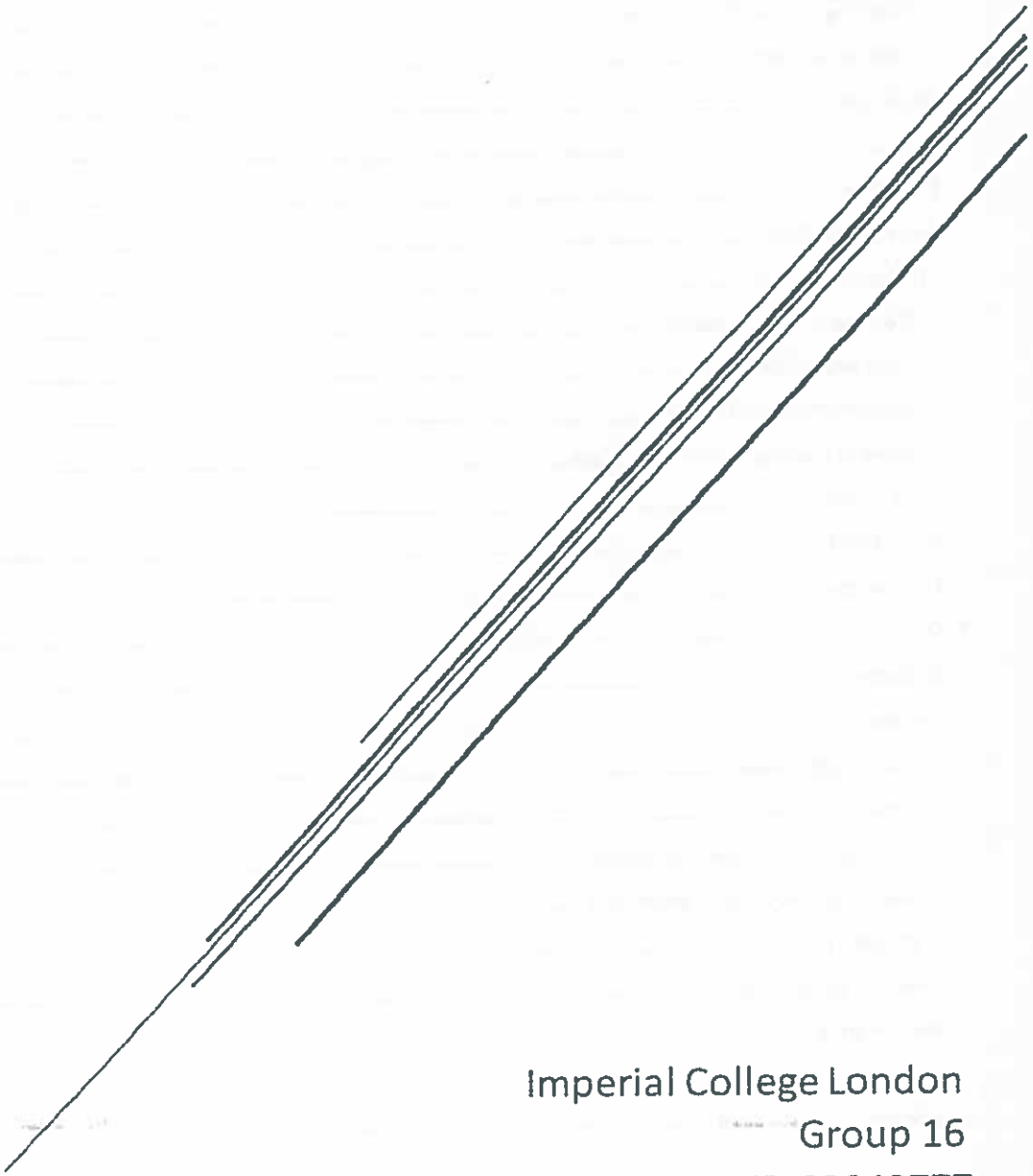
67%

– ok reports
– Easily results on task 8.

DIGITAL SYSTEM DESIGN

REPORT 3: TASK 6, 7 & 8



Imperial College London
Group 16
Deloris Owusu, CID 00942777
Mert Kaya, CID 00970695

Contents

Task 6 2
Objective 2
Timing 2
Custom Instructions 2
Addition Block 2
Subtraction Block 3
Multiplier Block 4
Quartus 5
Nios II 6
Checking functionality in Nios 6
Testing All Cases 7
Conclusion 8
Task 7 8
Objective 8
Theory of CORDIC 8
CORDIC in Quartus 11
Floating to Fixed Conversion 11
Floor and Subtraction 11
Implementation of Cosine 12
Fixed to Floating Point Conversion 14
Final CORDIC 14
Qsys & NIOS II 15
Conclusion 16
Task 8 17
Objective 17
Precision 17
Optimising the Code 17
Optimising Quartus 18
Floating to Fixed Point Conversion 18
Full Implementation in Hardware 19
Possible Improvements 20
Final Implementation 20
Performance Graph 21
Conclusion 21
References 22

Task 6

Objective

We will attempt to familiarise ourselves with the process of creating hardware blocks and adding custom instructions into our system, namely floating point addition, subtraction and multiplication, as well as analyse the resources and latency associated with this hardware compared to previous results.

Timing

When making the initial custom instruction we noticed a problem with our timing requirements and saw that our new design violated the timing constraints. Adding a custom instruction to our NIOS system resulted in significant changes to Qys. In task 3, when we created the phased locked loop we saw that a phase shift of about -46 degrees was needed for a 50MHz clock and that this would have to be tuned according to our DE0 board. Therefore we used a new value of 2.58ns instead of -2.55 and saw that the timing was no longer violated.

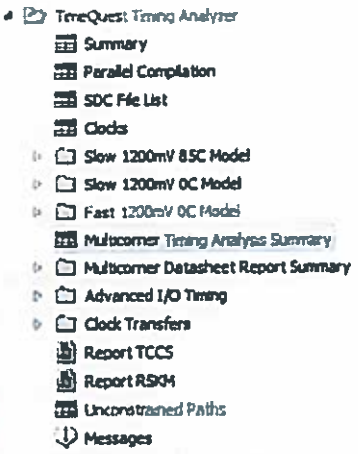


Figure 1 – Time analysis of our system

It is the slow model that must not be violated but the fast model is the worst case scenario, therefore it is not necessary to meet the timing constraints of the fast model.

Custom Instructions

After we finished the custom instruction tutorial and ensured all values were correct we proceeded to implement the addition, subtraction and multiplier blocks. (if have time find latency of each individual block by using the tick function gave us)

Addition Block

We implemented an addition block using the MegaWizard tool, where there were several design choice we needed to make. Firstly, the clock cycles was chosen to be 7, as this is the minimal amount possible for the MegaWizard function, and we are optimising for speed. We chose single precision because double precision would come at the expense of additional resources which are not necessary since our results for all three test cases are already quite accurate. Also, the addition block was optimised for speed and we decided not to place an asynchronous clear or a clock enable port because they are not necessary for our system and we are trying to make the block as efficient as possible with the fewest possible resources.

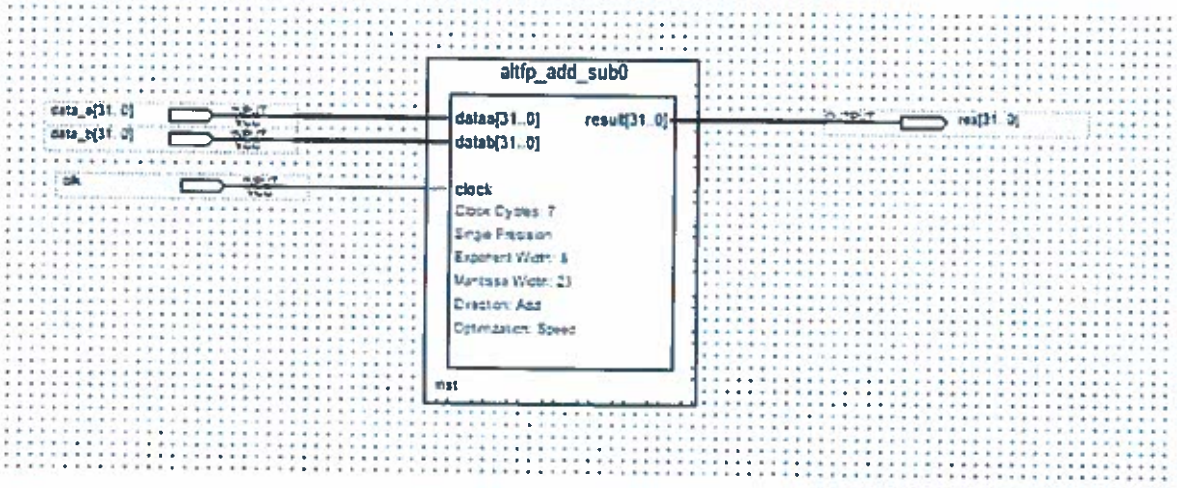


Figure 2 – Custom instruction addition block

We knew that the resources used in our overall architecture would increase, however, it only increases the resources by the following amount:

$$FPGA\ Resources = \frac{1}{3}\left(\frac{799}{15408} + \frac{0}{112} + \frac{36}{516096}\right) = 1.7309\%$$

This was significant increase in resources, however, we do expect to see a significant increase in latency so this a worthwhile trade-off.

In order to determine whether our adder behaved as expected we simulated the block in ModelSim, we tried two test cases and saw that our adder behaved as expected.

	Test 1	Test 2
Data A	21.5	30.75
	01000001101011000000000000000000	01000001111101100000000000000000
Data B	21.5	7.05
	01000001101011000000000000000000	01000000111000011001100110011010
Result	43	37.8
	01000010001011000000000000000000	01000010000101110011001100110011

As seen in figure 3, we obtained the result after 7 clock cycles as stated before.

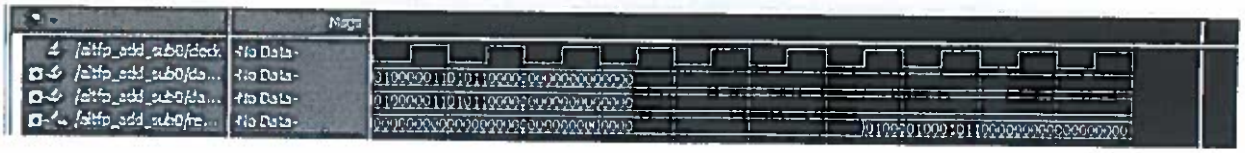


Figure 3 – Simulation of first test case in ModelSim

Subtraction Block

The subtraction block is almost identical to the addition block, but the subtraction only option was ticked, meaning it has a latency of 7 clock cycles and is also for speed and with single precision.

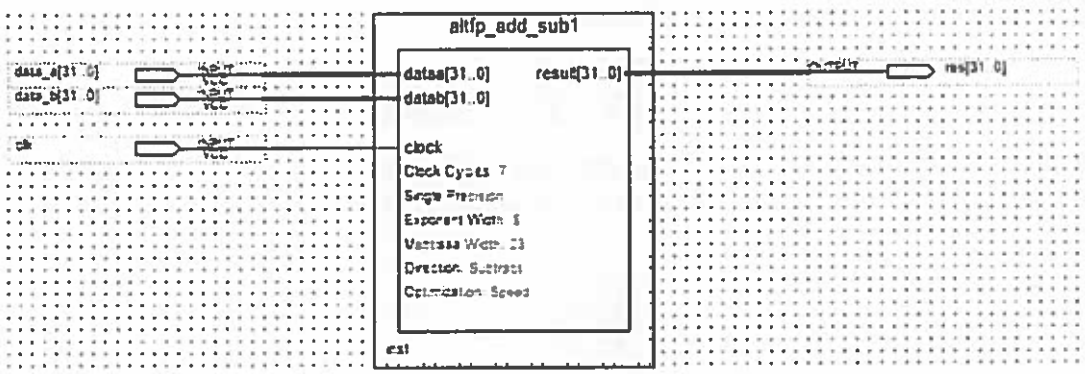


Figure 4 – Subtraction block

Again we saw the subtraction block take up a small but significant amount of resources in our system, which was very similar to the addition block but we again expected a worthwhile trade-off.

$$FPGA\ Resources = \frac{1}{3}\left(\frac{794}{15408} + \frac{0}{112} + \frac{36}{516096}\right) = 1.7201\%$$

Again in order to determine whether our subtraction block was functioning correctly, we modelled the block in ModelSim. We used the following two test cases and the block performed correctly.

	Test 1	Test 2
Data A	30.75	12.6
	01000001111101100000000000000000	01000001010010011001100110011010
Data B	7.05	29.55
	01000000111000011001100110011010	01000001111011000110011001100110
Output	23.7	-16.95
	01000001101111011001100110011010	11000001100001111001100110011010

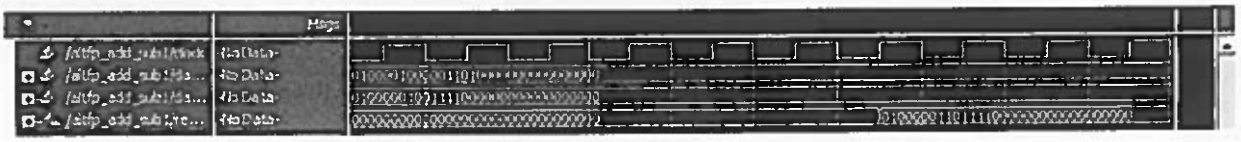


Figure 5 – Subtraction test 1 ModelSim

Multiplier Block

Lastly, we needed to implement a multiplier to complete our task. The block is shown in figure 6 where we chose the latency to be 5 because that was the minimum latency provided, again in a bid not waste resources we choose single precision and removed the clock enable and asynchronous inputs.

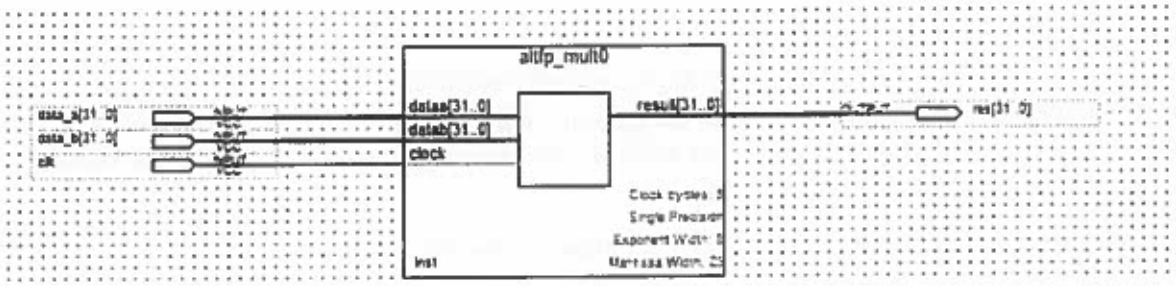


Figure 6 –Floating point multiplier block

The resources for the multiplier block is the following:

$$FPGA\ Resources = \frac{1}{3}\left(\frac{253}{15408} + \frac{7}{112} + \frac{0}{516096}\right) = 2.6307\%$$

The multiplier resources used was a lot greater than those of the addition and subtraction. When we were making the multiplier block we had an option to tick whether to use dedicated multiplier circuitry or not:



We presumed the resources used with the dedicated circuitry would be less and we were incorrect: not using the dedicated multiplier circuitry used fewer resources.

$$FPGA\ Resources = \frac{1}{3}\left(\frac{931}{15408} + \frac{0}{112} + \frac{0}{516096}\right) = 2.0141\%$$

This useed a smaller amount of resources than with the multiplier circuitry; however, looking back at task 5, we decided to use embedded multipliers instead of logic elements because of the speed. Also, since we will need to use the multiplier many times, implementing the embedded multipliers seemed like the wiser design choice. (maybe try in nios for ticks only if we have time)

Next we tested our multiplier with ModelSim to verify the results and we saw that they were correct for both positive and negative values.

	Test 1	Test 2
Data A	12.75	2.5
	01000001010011000000000000000000	01000000001000000000000000000000
Data B	12.75	-2.25
	01000001010011000000000000000000	11000000000100000000000000000000
Output	162.5625	5.625
	01000011001000101001000000000000	11000000101101000000000000000000

The multiplier calculates the results after 5 clock cycles which is faster than both the addition and subtraction block.

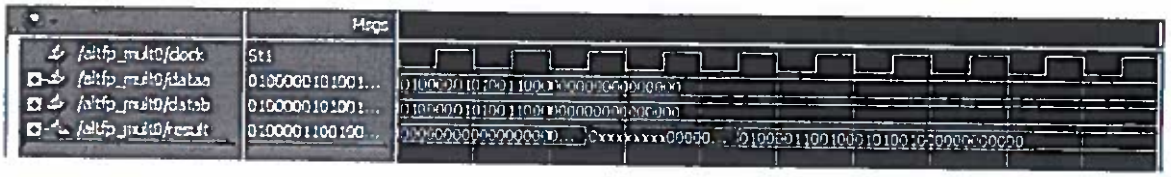


Figure 7 simulation of multiplier test case 1

Quartus

Once all the blocks were created in order to implement them into our system, we had to place them into Qsys. We did this my creating a new component, adding it to our NIOS system, generating and finally updating the blocks. We choose the clock cycles of each block to be the latency plus 1 so as to ensure the quickest possible execution.

We then recompiled the newly made system and determined the FPGA resources.

$$FPGA\ Resources = \frac{1}{3}\left(\frac{4950}{15408} + \frac{11}{112} + \frac{46{,}777}{516096}\right) = 17.0037\%$$

Revision Name	hello_world
Top-level Entity Name	hello_world
Family	Cyclone III
Device	EP3C16F484C6
Timing Models	Final
Total logic elements	4,950 / 15,408 (32 %)
Total combinational functions	4,540 / 15,408 (29 %)
Dedicated logic registers	3,097 / 15,408 (20 %)
Total registers	3165
Total pins	47 / 347 (14 %)
Total virtual pins	0
Total memory bits	46,777 / 516,096 (9 %)
Embedded Multiplier 9-bit elements	11 / 112 (10 %)

Figure 8 FPGA Resources

$$\frac{17.00 - 11.25}{17.00} = 33.8\%$$

Compared to the previous amount of resources, there was a 33.8% increase in resources; however, we predict that the decrease in latency (due to the implementation in hardware) will provide a good trade-off.

Nios II

First we looked into the system.h file and made sure our custom instructions were initialised. We saw that they were but not for floating point. The code that was present was "inii" where the *i* stands for integer, and so we needed to change this to "fnff" in order for our custom instruction to compute floating point calculations. We cannot change the system.h file and instead recreating a floating point version of the function with a different name at the beginning of every program where the custom instructions will be used as shown in figure 9.

```
#define ALT_CI_FP_ADDf_0(A,B) __builtin_custom_fnff(ALT_CI_FP_ADD_0_N,(A),(B))
#define ALT_CI_FP_ADDf_0_N 0x0
#define ALT_CI_FP_MULTf_0(A,B) __builtin_custom_fnff(ALT_CI_FP_MULT_0_N,(A),(B))
#define ALT_CI_FP_MULTf_0_N 0x2
#define ALT_CI_FP_SUBf_0(A,B) __builtin_custom_fnff(ALT_CI_FP_SUB_0_N,(A),(B))
#define ALT_CI_FP_SUBf_0_N 0x1
```

Figure 9 – Redefining custom instructions

Checking functionality in Nios

In order to further check the functionality of our blocks we test each case in NIOS II. We looked at the values from the addition, subtraction and multiplication blocks we designed in our code.

```
Floating point addition result: 37.29999923706054675
Floating point addition result from custom instr: 37.29999923706054675
Floating point subtraction result: 23.20000076293945125
Floating point subtraction result from custom instr: 23.20000076293945125
Floating point multiplication result: 8.739999771118164062
Floating point multiplication result from custom instr: 8.739999771118164062
```

Figure 10 – Output results from Nios

	Inputs		Result	MATLAB Result	Error
Addition Cases	A	13.55	37.29999923706	37.3	7.6×10^{-6}
	B	23.75			
	A	4.7	10.29999923706	10.3	7.6×10^{-6}
	B	5.6			
Subtraction Cases	A	30.75	23.20000076293	23.2	7.6×10^{-7}
	B	7.55			
	A	12.75	-6.850000381469	-6.85	3.8×10^{-7}
	B	19.6			
Multiplier Cases	A	2.5	5.625	5.625	0
	B	2.25			
	A	3.8	8.739999771118	8.74	2.3×10^{-7}
	B	2.3			

There was a very small error occurring which slightly varied depending on the block used, which was due to the fact the value was truncated and we chose single precision so the precision bits were limited. Noting the 0 result for the multiplication of the numbers 2.5 and 2.25 (answer of 5.625), we saw that numbers which could be perfectly represented by floating point notation—which are positive or negative powers of 2—had no error.

Testing All Cases

Next we looked at how the hardware affected the latency of our system. The three test cases are shown below.

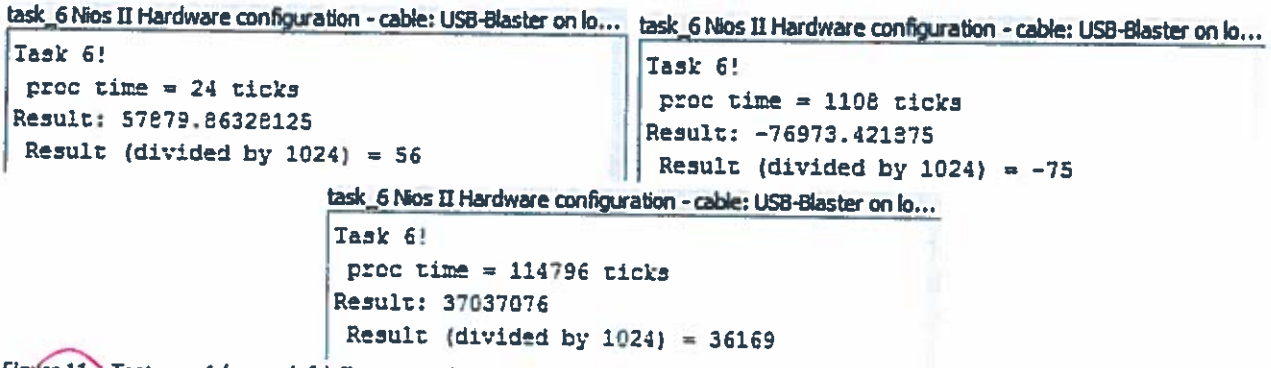


Figure 11 – Test case 1 (upper left), Test case 2 (upper right), Test case 3 (bottom)

The 96bit precision MATLAB results are the following:

Case 1	Case 2	Case 3
57879.8729731121	-76972.174963514	37022698.0051426

We implemented the function in the following way using our addition, subtraction and multiplier blocks.

```
l = cos(ALT_CI_FP_SUBf_0(floor(x[i] / 4), 32));
l = ALT_CI_FP_MULTf_0(l, x[i]);
l = ALT_CI_FP_MULTf_0(l, x[i]);
m = ALT_CI_FP_MULTf_0(x[i], 0.5);
y = ALT_CI_FP_ADDf_0(y, l);
y = ALT_CI_FP_ADDf_0(y, m);
```

The table below compares the previous task with the result we achieved.

	Old Version(Task 5)[1]			New Version (Task 6)		
Test Case	1	2	3	1	2	3
Error (%)	1.935×10^{-10}	1.935×10^{-10}	2.3488×10^{-6}	1.67447×10^{-6}	0.00161992	0.038882
Latency	27	1439	142483	24	1108	114796
Program Size(KBytes)	82	82	82	46	45	45

The error increased; however, the error percentage is still fairly insignificant even though there was a relatively large increase. This is a trade-off we are willing to accept because the percentage error relatively small and a large amount of precision is not required for this task. Furthermore, as stated above, the percentage error is due to the error seen above from the custom instructions but we are mainly looking at resources.

[1] Note that in task 5, due to the way the x values were initiated, the result was wrong. However, it is unclear to say whether the error would have been consistent. We also used float values instead of double which was wrong in task 5. This may affect the results.

The most significant change was the decrease in latency:

Case 1: $\frac{27-24}{27} = 11.1\%$ Case 2: $\frac{1439-1108}{1439} = 23.0\%$ Case 3: $\frac{142483-114796}{142483} = 19.4\%$

On average we saw approximately a 17% decrease in latency. Although there was a 33% increase in resources, the decrease in the latency made this a worthwhile trade-off.

There was also a decrease in program size because fewer software instructions were used.

Conclusion

This task took the first steps from implementing the function in hardware instead of software. We saw an increase in resources; nevertheless. the decrease in latency made it a worthwhile trade-off.

A

It would be easier to see the gains, if you had visualized the results with a digram.

Task 7

Objective

We will implement the CORDIC algorithm in place of the built-in cosine function we used in software. We are predict again a decrease in latency and an increase in resources.

Theory of CORDIC

We used an algorithm called Coordinate Rotation Digital Computer (CORDIC) in order to implement the cosine function quickly in our hardware. It is an algorithm that, as its name suggests, rotates the phase of a complex set of coordinates within the unit circle until the desired angle is approximately reached. Mathematically, it only requires addition, subtraction and multiplying by powers of 2 (shifting bits), making it ideal for fast hardware implementation.

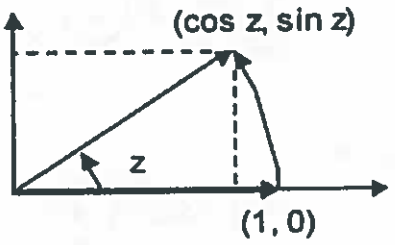


Figure 12 – Angle diagram

We first start with the coordinates (1, 0). As shown in figure x, the phase is rotated by an angle. Calling this angle θ, the old coordinate (X_o, Y_o), and the new coordinate (X_n, Y_n), the general form of the rotation can be written as:

$$X_n = X_o \cos(\theta) - Y_0 \sin(\theta) \quad (1)$$

$$Y_n = X_o \sin(\theta) + Y_o \cos(\theta)$$

To reach the desired angle, an infinite amount of rotations is required. However, after several iterations the output of the CORDIC algorithm is approximately equal to the desired angle and since we only have a possible 32 bits of precision then, then we only need a certain amount of iterations. The number of bits is actually proportional the number of iterations therefore to gain full precision we need 32 iterations. In practice fewer are required and this will be discussed later.

If equation 1 is presented in matrix notation, the following is planar rotation of a vector which is exactly what the CORDIC algorithm does:

$$\begin{bmatrix} X_n \\ Y_n \end{bmatrix} = \begin{bmatrix} \cos\theta & \sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} X_o \\ Y_o \end{bmatrix} \quad (2)$$

It is easier to perform the calculations in matrix notation hence why we have transformed equations 1 into 2. The angle θ can be calculated in multiple steps, so we will define a single iteration by i and a single step by the following:

$$\begin{bmatrix} X_{i+1} \\ Y_{i+1} \end{bmatrix} = \begin{bmatrix} \cos\theta_i & \sin\theta_i \\ \sin\theta_i & \cos\theta_i \end{bmatrix} \begin{bmatrix} X_i \\ Y_i \end{bmatrix} \quad (3)$$

The goal of the manipulation is to have no trigonometric functions or multiplies within the equation so our first step is to take $\cos\theta_i$ as factor:

$$\begin{bmatrix} X_{i+1} \\ Y_{i+1} \end{bmatrix} = \cos\theta_i \begin{bmatrix} 1 & -\tan\theta_i \\ \tan\theta_i & 1 \end{bmatrix} \begin{bmatrix} X_i \\ Y_i \end{bmatrix} \quad (4)$$

Now this equation has one fewer multiply than the one in equation 3.

The other trick that the CORDIC algorithm uses is to only use values of the angle such that the tangent of a step is a power of 2.

$$\theta_i = \arctan\left(\frac{1}{2^i}\right) \quad (5)$$

θ_i is the angle for each step. After progressing through each iteration we should get a sum which approximately equals the final angle of rotation θ. The sum is shown as the following:

$$\sum_{i=0}^{\infty} S_i\theta_i = \theta \quad (6)$$

$S_i = \{-1, +1\}$

This means that the tangent values are:

$$\tan\theta_i = S_i 2^{-i} \quad (7)$$

So if we input this value for *tan* back into equation 4 we get the result:

$$\begin{bmatrix} X_{i+1} \\ Y_{i+1} \end{bmatrix} = \cos\theta_i \begin{bmatrix} 1 & -S_i 2^{-i} \\ S_i 2^{-i} & 1 \end{bmatrix} \begin{bmatrix} X_i \\ Y_i \end{bmatrix} \quad (8)$$

Now we only need to calculate the cosine function and perform some shifts and additions, which is a dramatic improvement from the initial result. However, we would still need to calculate the cosine function, so in order to completely eliminate it we can rewrite the cosine function as the following:

$$\cos\theta_i = \cos\left(\tan^{-1}\left(\frac{1}{2^i}\right)\right) \quad (9)$$

If we compute this for every single i value, we will just be multiplying the results and this coefficient is traditionally referred to as K.

$$K = \prod_{i=0}^{n-1}\cos\left(\tan^{-1}\left(\frac{1}{2^i}\right)\right) = \lim_{n\to\infty}\prod_{i=0}^{n-1}\cos\left(\tan^{-1}\left(\frac{1}{2^i}\right)\right) \approx 0.607252935\ldots \quad (10)$$

K is the same for all the vectors and for all the values of the angle θ.

Looking back at equation 1 we can now fully compute it since K is computed before we can take this into account at the later stages. The final matrix form is:

$$\begin{bmatrix} X_{i+1} \\ Y_{i+1} \end{bmatrix} = \begin{bmatrix} 1 & -S_i 2^{-i} \\ S_i 2^{-i} & 1 \end{bmatrix} \begin{bmatrix} X_i \\ Y_i \end{bmatrix} \quad (11)$$

This can also be written as:

$$X_{i+1} = X_i - S_i 2^{-2i} Y_i$$
$$Y_{i+1} = Y_i - S_i 2^{-2i} X_i \quad (12)$$

However, this is not the final result. We define a new variable Z which is defined as the part of θ which has not been rotated yet.

$$Z_{i+1} = \theta - \sum_{i=0}^{n}\theta_i \quad (13)$$

For each iteration, S_i is equivalent to the sign of Z_i, therefore $S_i = sign(Z_i)$. If we use this fact and combine it with equation 6 we get a result such that the part of the angle which has not been rotated converges to 0.

Next we will present the pseudo-code for writing the algorithm.

CORDIC_Z

The initial conditions required are:

$X_i = 0.607252935$
$Y_{i+1} = 0$

$Z_{i+1} = \theta$

For i = 0 to n
If $Z_i >= 0$

$$X_{i+1} = X_i - 2^{-2i} Y_i$$
$$Y_{i+1} = Y_i + 2^{-2i} X_i$$

$$Z_{i+1} = Z_i + \tan^{-1}\left(\frac{1}{2^i}\right)$$

Else

$$X_{i+1} = X_i + 2^{-2i} Y_i$$
$$Y_{i+1} = Y_i - 2^{-2i} X_i$$

$$Z_{i+1} = Z_i + \tan^{-1}\left(\frac{1}{2^i}\right)$$

End for loop

Then return

$X_n = \cos\theta$
$Y_n = \sin\theta$

$Z_n = 0$

It requires a lookup table of the values $\tan^{-1}\left(\frac{1}{2^i}\right)$ up to the number of iterations n. As stated before, the number of iterations is proportional to the number of bits, meaning that there is 1 iteration per bit, so the required look up table would be relatively small and comprising only of the number of bits.

CORDIC in Quartus

The function we are required to implement using the CORDIC algorithm is:

$$f(x) = \cos(floor\left(\frac{x_i}{4}\right) - 32)$$

There are several steps to implementing the CORDIC algorithm in Quartus. We first have to do the inner part of the cos function which is the division, floor and subtraction. This is will all be done with the CORDIC block so first we will implement this.

Floating to Fixed Conversion

The first thing to do to convert floating to fixed point. However, in order to implement all the individual operations relatively easily, a fixed point representation is preferred to floating point since the CORDIC function uses two's complement notation and binary shifts. There is already a built in function to carry out the conversion and using MegaWizard and so we made the block. The fixed point properties we chose were to have a 16 bits for the custom number and 16 bits for the fractional part, which came from the simple fact the only values we can have inside the cosine are the integer values from -31 to 32, so this value does not need to have high precision.

In order to test the conversion, we simulated it in ModelSim for multiple values. Figure 12 shows an example using 67.45 which showed that converter was working correctly.



Figure 13 – Floating to Fixed converter for 67.45

Note here that the max possible value that will be entered is 6 bits so there are some redundant bits but because of the 32-bit floating point notation they are necessary.

Floor and Subtraction

$$floor\left(\frac{x_i}{4}\right) - 32$$

This is the function that gets fed into the cosine. Implementing this function in Verilog is quite simple but first we have to discuss what the floor function is. The floor function essentially removes the fractional part of a number and returns just the integer value. Since the custom part of our fixed point integer is 16 bits and the fractional part is 16 bits we have to get rid of this 16 LSB fractional part. This entails shifting the input 16 bits to the right. The division by 4 is again a shift to the right 2 times.

Altogether this makes 18 shifts to the right. The only part left is just subtracting by 32 which is simple to do in Verilog. We analysed the current file to verify it had no errors and it is shown in the figure below.

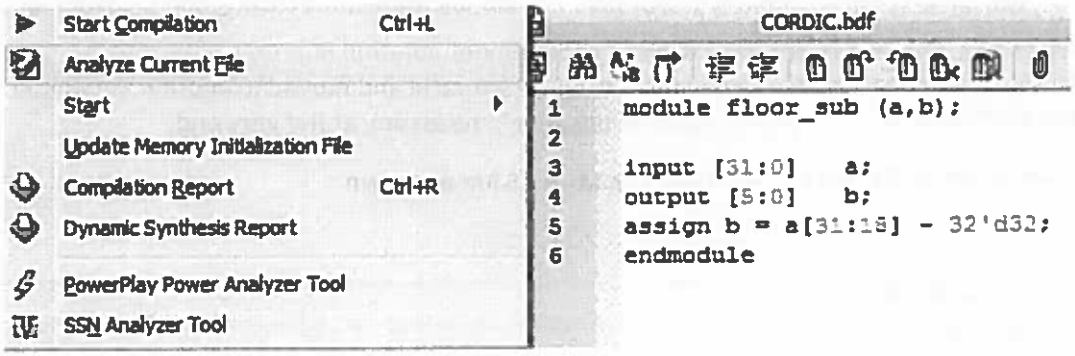


Figure 14 – Floor and subtraction in one function

The floor functions we tested are shown below:

$$floor\left(\frac{90}{4}\right) - 32 = -10$$

$$floor\left(\frac{203.45}{4}\right) - 32 = 18$$

As we can see from the simulation below we obtained the correct results:



Figure 15 – ModelSim for floor function

Implementation of Cosine

After implementing the floor function, we implemented the CORDIC algorithm. This was split into two blocks: the LUT and the rotations.

The CORDIC algorithm requires a LUT to store the values of -32 to 31 in radians for the rotation calculations all the values from -31 to 32 are shown in figure 16.

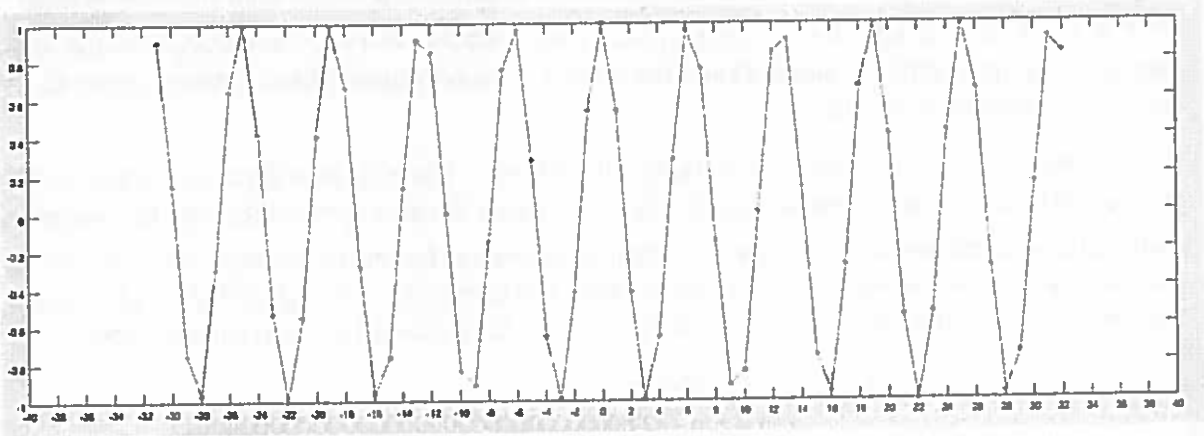


Figure 16 – MATLAB plot of values from -31 to 32

These were found in MATLAB and then converted to a fixed point value. Unlike before, where we had 16 custom and 16 fraction bits, we decided to use 1 sign, 1 custom and 30 fraction bits. This is because our CORDIC algorithm will only work in the first quadrant, meaning that the values of the inputs range from 0 to $\frac{\pi}{2}$; the latter is approximately 1.57079633 and can be represented using only 1 custom bit and 2 fraction bits. The sign is not used in two's complement notation; it is only a bit representing whether the cosine will be positive or negative. It will be stored in and moved to another variable in the rotation block and will convert the angle to negative if necessary at the very end.

The code used to generate the radians values in MATLAB are as shown:

```
rad_mafe.m
for i = -32:31
    % take module to convert
    rad(i + 33) = mod(i, 2*pi);
    % assign the sign with default of zero
    sign(i + 33) = zeros;
    % check if it is in second quadrant and if so put in first quadrant
    % with negative sign
    if rad(i + 33) > pi / 2 && rad(i + 33) <= pi
        rad(i + 33) = pi - rad(i + 33);
        sign(i + 33) = 1;
    % check if it is in third quadrant and if so put in first quadrant
    % with negative sign
    elseif rad(i + 33) > pi && rad(i + 33) <= 3 * pi / 2
        rad(i + 33) = rad(i + 33) - pi;
        sign(i + 33) = 1;
    % check if it is in fourth quadrant and if so put in first quadrant
    % with no sign change
    elseif rad(i + 33) > 3 * pi / 2 && rad(i + 33) <= 2 * pi
        rad(i + 33) = 2 * pi - rad(i + 33);
    % check if it is in first quadrant and if so make no changes
    else
    end
end
```

Figure 17 – MATLAB code to generate the values

By identifying which quadrant the number is in, the conversion to positive radians values is straightforward and the signs are stored in a different array (similar to our storage in the LUTs).

After writing the radians values into the LUT, we input the selected one into the CORDIC rotation block. With every change of input Z_i, the rotation function begins. From there, the sign bit is written to another variable and then changed to 0. The variables X_i and Y_i are initialised as 0.607252935009250 and 0 respectively. Then the rotation begins. The input Z_i is set to be initially positive (MSB set to 0) and so $\frac{\pi}{4}$ is subtracted from it and the X_i and Y_i values are updated accordingly as described above. From there, each iteration is checked to be positive or negative: if positive, right shift of $\frac{\pi}{4*2^i}$ (right shifts of i) are subtracted and vice versa. X_i and Y_i are updated after each iteration. After the last iteration, the sign variable is checked and if the value is 1, the negative of the calculated output $X_n \approx \cos(\theta)$ is used as the final output.

We decided to make the function very precise initially and so made 30 iterations since we have 30 fraction bits and so $\frac{\pi}{4}$ can be shifted a maximum of 29 times. We also unrolled the loop to make the code faster: with this we can eliminate the conditional execution for the first iteration and by avoiding the use of a for or while loop, we were able to have a fast combinatorial program that is not clocked and hence is faster than if it was clocked. A sample code for the iterations is as shown below:

```
if(z_n[31] == 1'b0) begin
        z_n = z_n - 32'b00000000000000000000000000000011;
        x_n = x_o - (y_o >> 28);
        y_n = y_o + (x_o >> 28);
```

```
end
else begin
        z_n = z_n + 32'b00000000000000000000000000000011;
        x_n = x_o + (y_o >> 28);
        y_n = y_o - (x_o >> 28);
end
x_o = x_n;
y_o = y_n;
if(z_n[31] == 1'b0) begin
        z_n = z_n - 32'b00000000000000000000000000000001;
        x_n = x_o - (y_o >> 29);
        y_n = y_o + (x_o >> 29);
end
else begin
        z_n = z_n + 32'b00000000000000000000000000000001;
        x_n = x_o + (y_o >> 29);
        y_n = y_o - (x_o >> 29);
end
x_o = x_n;
y_o = y_n;*/
// choose output to be negative or positive according to the sign
if (sign == 1'b0) begin
        angle = x_o;
end
else begin
        angle = -x_o;
end
```

Fixed to Floating Point Conversion

Now that CORDIC is functioning correctly, we can now implement the last part which is just the fixed to floating point converter. We already implemented a floating to fixed point converter and we chose to use the same MegaWizard function, except this time we chose the option as fixed to floating point. Furthermore, we also changed the number of custom bits as discussed above. We used 1 bit for the sign, 1 bit for the custom part and the rest of the 30 bits for precision. The output of the CORDIC rotation code is in two's complement and therefore we used 2 custom bits and 30 precision bits for this converter, which was different from our initial converter.

This converter had a latency of 7 cycles, and together with the final part the whole CORDIC block was complete.

Final CORDIC

The final top level CORDIC block diagram is shown below in figure 18 and is comprised of all the individual elements we put together.

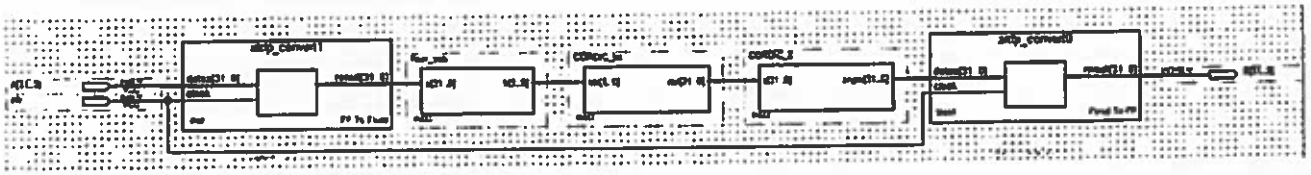


Figure 18 – Top level of cosine

The resources allocated for the CORDIC diagram was the following:

$$FPGA\ Resources = \frac{1}{3}\left(\frac{4742}{15408} + \frac{0}{112} + \frac{0}{516096}\right) = 10.25\%$$

This is a relatively large number of resources to add to our system, however, the desired effects of a decrease in latency would be an excellent compromise.

Next we simulated the whole CORDIC top level function on ModelSim. To check whether it functioned correctly we input the following values (out of the many we simulated). We have only presented these as an example:

Input	255	01000011011111110000000000000000
Result	0.9147419	00111111011010100010110010000110
MATLAB Result	0.914742357804531	
Error	4.578×10^{-7}	
Input	60	01000010011100000000000000000000
Result	-0.2751621	10111110100011001110001000001100
MATLAB Result	-0.275163338051597	
Error	1.2638×10^{-6}	
Input	97.45	01000010110000101110011001100110
Result	-0.14549877	10111110000101001111110110100001
MATLAB Result	-0.145500033808614	
Error	1.2638×10^{-6}	

We wanted to test all the values in order to check if the function was working correctly. The error was relatively small and later on we will elaborate on exact source of the error. Figure 19 below shows the ModelSim simulation diagram.



Figure 19 – ModelSim of CORDIC top level

We observed that it took 14 clock cycles for the correct output to emerge. This is because 2 delays of 7 clock cycles come from each converter and the other (CORDIC) blocks are combinatorial functions.

Qsys & NIOS II

Once we verified it was working in ModelSim we then began to implement the system in to Qsys. Some problems arose here, but we did not realise this until we implemented it in NIOS. Since the latency of or floating point to fixed point converter was 7 for the CORDIC system we gave it a latency of 8; however, when we began to run the test in NIOS the output had unusually large values. The problem was the clock: we had not taken into account that there were two converters, one float to fixed and the other fixed to float, therefore the only value the output was ever calculating was the initial output of 0, which gives out a value 0.8342... and this was the only value being returned from the fixed to floating point converter since there were not enough clock cycles for it to output the right result. We accidently overlooked our ModelSim diagram and this cause an error in our results. This mishap resulted in us being far more careful of clock cycles and making sure to double check with ModelSim and carefully calculate them as this was an unfortunate oversight. We calculated the clock cycles in the following way:

7(altfp_converter0) + 7(altfp_converter1) = 14 clock cycles

We then knew to set the custom block clock cycles to the above clock cycles + 1. Consequently for the input of our Qsys we set the clock cycles to 16 which output the correct result. The results are shown below[2]:

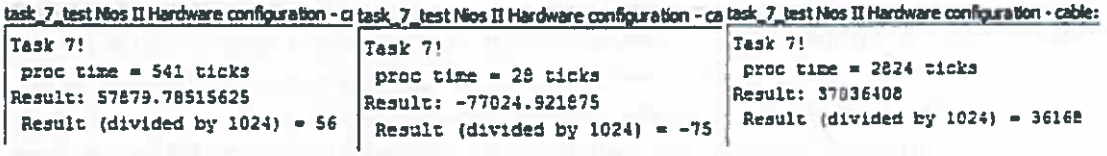


Figure 20 - CORDIC Implementation in NIOS

The MATLAB results are the following:

Case 1	Case 2	Case 3
57879.8729731121	-76972.174963514	37022698.0051426

We will compile the results in a table in comparison with the previous design in task 6. The resources used in this task is the following:

$$FPGA\ Resources = \frac{1}{3}\left(\frac{9837}{15408} + \frac{11}{112} + \frac{46{,}804}{516096}\right) = 27.58\%$$

$$\frac{27.58 - 17.00}{27.58} = 38.36\%$$

The resources increased by about 10%. This was consistent with the resources of the CORDIC function we implemented. A 38.36% percentage increase in resources, however, is quite large but as shown in the table below the latency has drastically decreased.

	Old Version(Task 6)			New Version (Task 7)		
Test Case	1	2	3	1	2	3
Error (%)	1.67447×10^{-6}	0.00161992	0.038882	1.5172×10^{-4}	0.06848	0.03702
Latency	24	1108	114796	0.541	28	2824
Program Size(KBytes)	46	46	46	36	36	36

The error was larger yet still of the small magnitude of 10^{-6}. We will later try and improve the precision in task 8. The error arises from the CORDIC function and we will see later how this error accumulates and causes a small but noticeable error.

The latency decreased greatly:

Case 1: $\frac{24-0.541}{24} = 97.75\%$ Case 2: $\frac{1108-28}{1108} = 97.47\%$ Case 3: $\frac{114796-2824}{114796} = 97.54\%$

The percentage decreases in latency were large. Calculating the cosine function in the hardware was quite expensive in terms of resources, but by implementing it we managed to reduce the latency by an average of around 97.5% which was definitely worth the resources-latency trade-off.

Conclusion

In this task, we saw how effective adding hardware could be in decreasing the latency. We also learned about the importance of timing and synchronising blocks.

[2] Note for test case 1 we ran the program 1000 times to get a more accurate value for ticks, hence it seems abnormally large for test case one.

Task 8

Objective

We will try optimise our system using different methods. We will aim to find the right trade-off between amount of resources and the best latency.

There are many ways of optimising our system. We thought of several ideas, some of which we were able to implement and some, due to time constraints and debugging, we were not. Here we had the experience of creating, testing and debugging our own improved program. We have broken this part of our report down into two subsections: optimising the code (software) and optimising the hardware.

Precision

The implementation of our CORDIC algorithm caused there to be a greater error than in the built-in cosine function. We found the values for each CORDIC output by debugging in NIOS and setting our output variable y to be equal to the CORDIC outputs only. By adding a breakpoint in the for loop of generateVector, we were able to quickly access these values and compare them to a built-in cosine.

The errors were on average in the magnitude of 10^{-7}. The error for the CORDIC algorithm can be approximated by the function $\text{atan}\left(\frac{1}{2^{30}}\right) \approx 9.31322575 * 10^{-10}$.

The errors, when accumulated after several implementations of the CORDIC algorithm in the for loop, result in a larger error than before. However, the error was still very small, as shown in test case 3 where it was in the magnitude of 10^{-4}. However, in test case 3 the CORDIC algorithm caused the function to be over 30 times faster than before. Due to this, we decided that the precision error was a good trade-off for the increase in computational speed.

We ran the program with the 20-iteration CORDIC algorithm and found that the resources used decreased as follows:

$$FPGA\ Resources = \frac{1}{3}\left(\frac{8701}{15408} + \frac{11}{112} + \frac{46{,}804}{516096}\right) = 25.12\%$$

The number of ticks stayed relatively the same and the accuracy did not worsen significantly, therefore we decided use the 20 iteration version of CORDIC instead of our original 30 iteration one.

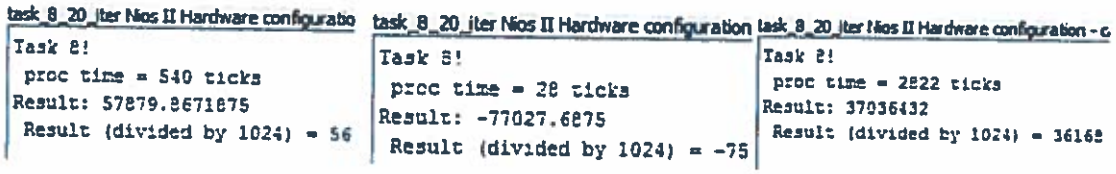


Figure 21 – CORDIC with 20 iterations

Optimising the Code

Now that we have implemented the design in hardware, we will first look at optimising the code.

We optimised the code by nesting all our hardware addition, multiplication and cosine functions into one code line as follows:

```
y = ALT_CI_FP_ADDf_0(ALT_CI_FP_ADDf_0(y, ALT_CI_FP_MULTf_0(x[i], 0.5)),
ALT_CI_FP_MULTf_0(ALT_CI_FP_MULTf_0(ALT_CI_CORDICf_0(x[i]), x[i]), x[i]));
```

This decreased the execution time because the program takes time to instantiate our two new variables in the generate function and assign values to them. The changes are as follows:

	Old Version(Task 7)			New Version (Task 8)		
Test Case	1	2	3	1	2	3
Latency	0.541	28	2824	0.375	21	2038

The change in latency is very significant:

Case 1: $\frac{0.541-0.375}{0.375} = 44.27\%$ Case 2: $\frac{28-21}{28} = 25.00\%$ Case 3: $\frac{2824-2038}{2824} = 27.83\%$

We see here that just a simple change in code can decrease our latency.

Optimising Quartus

Floating to Fixed Point Conversion

The way in which we convert from floating point to fixed point is quite simple due to the formatting of the floating point:

$$floating\ point\ number = \pm mantissa \times 2^{exponent}$$

Sign	Exponent	Mantissa
+1	2^2	1.25
0	129	2097152

Figure 22 – Example of floating point value 5.0

The code below performs the same function. The mantissa of the function is expressed as the first 23 bits of x and the exponent is assigned bits 30 to 23. Then the mantissa is shifted by the exponent minus 127, which is shown in how the conversion to floating point works. Then the sign is taken care of. We also decided to compute the floor and subtraction in this function considering the fact that since it is only one line, it will be much more efficient to calculate everything in this function.

```
always @(x) begin

        mantissa[22:0] = x[22:0];
        exponent=x[30:23];
        mantissa[23] = 1'b1;
        mantissa[31:24] = 0;
        temp= mantissa << (exponent-8'd127);

        if (x[31] == 1'b0)
                temp[31] = 1'b0;
        else
                temp[31] = 1'b0;
        temp= temp >> 7;
        fix = (temp>>18) - 32'd32;

end
```

As a result of implementing the floating point to fixed point converter, we have seen a decrease in resources:

$$FPGA\ Resources = \frac{1}{3}\left(\frac{3179}{15408} + \frac{0}{112} + \frac{0}{516096}\right) = 6.8773\%$$

This is the total resources of the new CORDIC function with the new converter. There is over a 3% drop in resources. Although the changing this did not significantly change the latency, it did change the resources which means it is a worthwhile improvement.

test1 Nios II Hardware configuration - cable:	test1 Nios II Hardware configuration - cable: USB-E	test1 Nios II Hardware configuration - cable: USB-E
Task 8! proc time = 369 ticks Result: 57879.8671875 Result (divided by 1024) = 56	Task 8! proc time = 20 ticks Result: -76980.71875 Result (divided by 1024) = -75	Task 8! proc time = 2011 ticks Result: 37036432 Result (divided by 1024) = 36168

The final resources after this converter decrease as shown:

$$FPGA\ Resources = \frac{1}{3}\left(\frac{8256}{15408} + \frac{11}{112} + \frac{46798}{516096}\right) = 24.1572\%$$

Full Implementation in Hardware

So far, we used different custom instructions to implement the adder, CORDIC and multiplier. Another possible way that might decrease the latency is to implement it all in synchronised hardware.

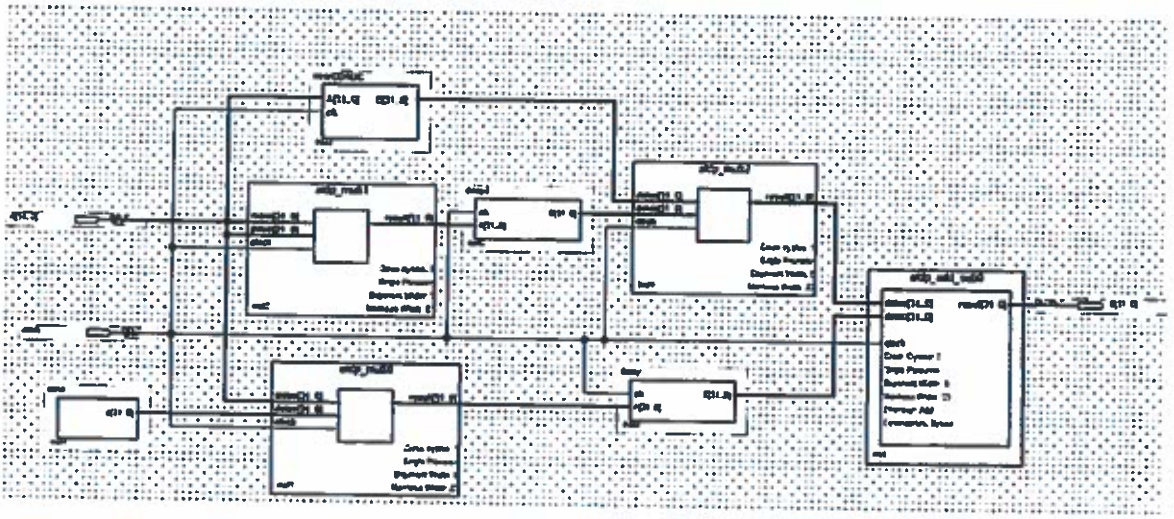


Figure 23 – Full hardware implementation block diagram

We now have three multipliers and an adder within the hardware performing the whole function. We used a small constant function to define the 0.5 and we used delays so that our system is synchronous and all the correct outputs reach the blocks at the same time. This may not be necessary but it means we have a safer and more reliable system. These are the results we obtained from ModelSim:

Input	76	01000010100110000000000000000000
Result	5279.411	01000101101001001111101101001010
MATLAB Result	5279.41260965633	
Error	4.578×10^{-7}	
Input	133.5	01000011000001011000000000000000
Result	9696.129	01000110000101111000000010000100
MATLAB Result	9696.15277075845	
Error	1.2638×10^{-6}	
Input	4.3	01000000100010011001100110011010
Result	19.06359	01000001100110001000001000111011
MATLAB Result	19.0635861958058	
Error	1.2638×10^{-6}	



Figure 24 – Full hardware implementation on ModelSim

As shown on ModelSim, it takes 17 clock cycles to get the correct result, therefore in Qsys we will set the number of clock cycles to 18.

We implemented it in NIOS and saw over a 50% improvement in latency for each case:

	Old Version(Not full function in hardware)			New Version (Full function in hardware)		
Test Case	1	2	3	1	2	3
Latency	0.369	20	2011	0.179	9	894

Case 1: $\frac{0.369-0.179}{0.369} = 51.49\%$ Case 2: $\frac{20-9}{20} = 55.00\%$ Case 3: $\frac{2011-894}{2011} = 55.54\%$

Possible Improvements

A simple yet effective method of hardware acceleration is multiplying the resources used by an integer N, which means having N inputs. This means that multiple x[i] values can be input and calculated in parallel; due to this, the computational speed would decrease by a factor of N. While we have implemented it, we did did not test this due to time constraints. Also, resource constraints would have limited us to only N = 2. We implemented some of our design combinatorial, however the end goal is to get rid of the for loop in the software and push in our x values, one by one combinatorial. This would require dynamic access memory and many other things which we were not capable of creating.

Another method of increasing the latency of the function is adding a phased lock loop to increase the clock frequency. If there are no (slow) timing violations, this should in theory increase the speed of our program by $\frac{clk_{new}}{clk_{old}}$, where clk_{old} = 50MHz. This again was not implemented due to time constraints.

If enough time was given for this project, we would have implemented these two designs and researched other methods for improving performance.

Final Implementation

This final implementation shows multiplying the N values by 2 like said above, since we were not able to run it on NIOS but this would be our next step in implementing the design was it not for time constraints.

B

→ But how can you provide N inputs?

– A wire design is in place (N = 2) but not working.

– Some optimisation in place, but not DMA

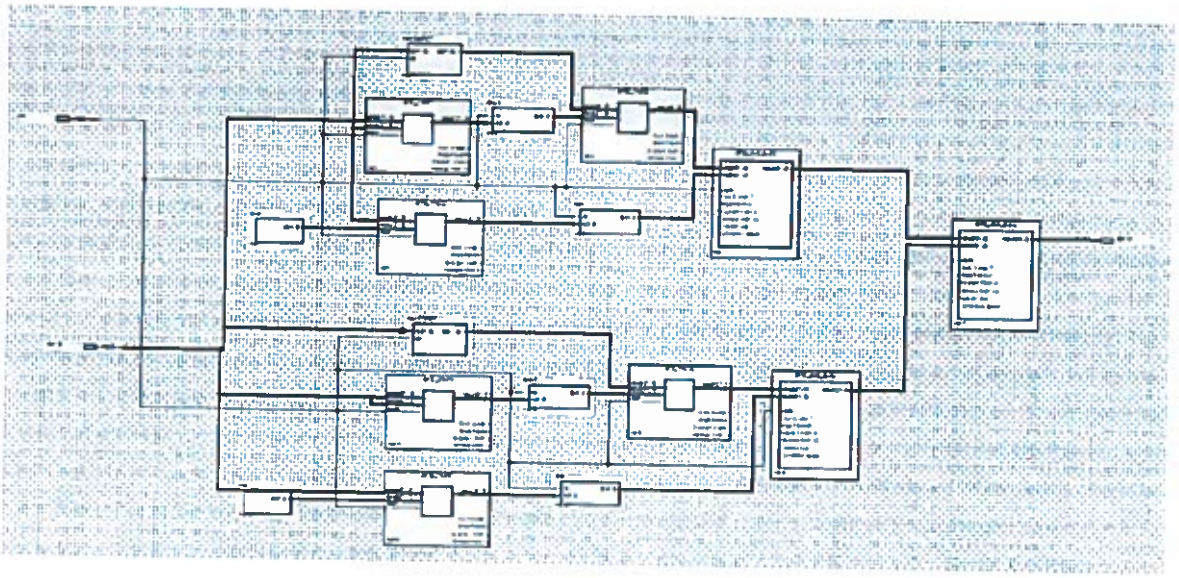
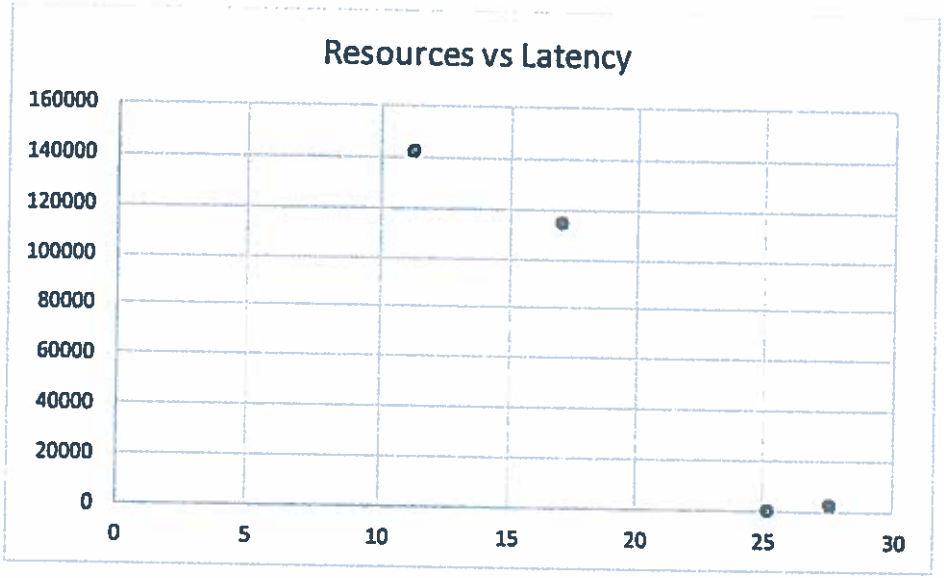


Figure 25 – Final Block Diagram

Performance Graph

Here shows a comparison graph without previous designs for case 3, the first dot on the left is task 5 and each dot the right is each subsequent task.



As we can see have managed to reduce latency massively by implementing the whole design in hardware.

Conclusion

After implementing multiple design strategies and making numerous tweaks to our system, we have found that the most effective methods for improving methods have been the hardware methods, especially ones that prioritise parallelisation. This is because software methods are limited by the hardware in which it is implemented.

References

[1]

https://www.h-schmidt.net/FloatConverter/IEEE754.html

[2]

http://opencores.org/websvn,filedetails?repname=cordic&path=%2Fcordic%2Ftrunk%2Fdocumentation%2Fcordic.pdf